

The AI Engineer's System Design Interview Guide

End-to-End RAG, LLM Systems, Evaluation,
Hallucination Reduction, and Production Architecture

VOLUME 1



AUTHOR

Lamhot Siagian

LINKEDIN NEWSLETTER
AI Engineering Insider

Contents

Preface	3
1 RAG Fundamentals	4
1.1 Core Concepts	4
1.1.1 RAG vs Fine-tuning vs Long-context LLMs	4
1.1.2 Embeddings: Dense vs Sparse vs Hybrid	5
1.1.3 Chunking Strategies	5
1.1.4 Vector Databases	5
1.1.5 Similarity Search: Cosine vs Dot Product	5
1.2 Tough Questions	5
1.2.1 Why use RAG instead of fine-tuning? When does RAG fail?	6
1.2.2 How do embedding models impact retrieval quality?	6
1.2.3 What happens if chunk size is too large or too small?	7
1.2.4 Explain cosine similarity vs inner product in high-dimensional space	7
1.2.5 How do you handle domain-specific vocabulary?	8
2 Advanced Retrieval Design	9
2.1 Core Concepts	9
2.1.1 Hybrid Retrieval: BM25 + Vector Search	9
2.1.2 Query Rewriting and Expansion	10
2.1.3 Re-ranking Pipeline	10
2.1.4 Multi-hop Retrieval	10
2.2 Tough Questions	10
2.2.1 Design a hybrid retrieval system end-to-end	11
2.2.2 Why is BM25 + vector better than vector alone?	11
2.2.3 How does re-ranking improve precision? Trade-offs?	12

2.2.4	How would you handle ambiguous queries?	13
2.2.5	How do you implement multi-hop reasoning retrieval?	13
3	RAG Architecture (Production-Level)	15
3.1	Core Concepts	15
3.1.1	End-to-End Pipeline	15
3.1.2	Stateless vs Stateful RAG	15
3.1.3	Agentic RAG: Planner – Executor – Verifier	16
3.1.4	Tool Calling	16
3.2	Tough Questions	16
3.2.1	Design a production RAG system for millions of users	16
3.2.2	How do you reduce latency in RAG?	17
3.2.3	How do you cache results effectively?	17
3.2.4	How do you handle failures in retrieval or LLM?	18
3.2.5	How would you design RAG for real-time vs batch use cases?	19
4	Evaluation & Metrics	20
4.1	Core Concepts	20
4.1.1	Retrieval Metrics	20
4.1.2	Generation Metrics (RAGAS)	20
4.1.3	Evaluation Tools	21
4.2	Tough Questions	21
4.2.1	How do you measure hallucination?	21
4.2.2	What is faithfulness in RAG?	22
4.2.3	How do you evaluate retrieval vs generation separately?	22
4.2.4	How would you build an offline evaluation pipeline?	23
4.2.5	How do you create a golden dataset?	24
5	Hallucination & Reliability	26
5.1	Core Concepts	26
5.1.1	Grounded Generation	26
5.1.2	Citation Enforcement	26
5.1.3	Confidence Scoring and Refusal	27
5.2	Tough Questions	27
5.2.1	How do you reduce hallucination in RAG?	27
5.2.2	What if retrieved documents are wrong?	28
5.2.3	How do you enforce citations in generated answers?	28
5.2.4	How do you detect low-confidence answers?	29
5.2.5	When should the system refuse to answer?	29

6	Performance & Scaling	31
6.1	Core Concepts	31
6.1.1	Approximate Nearest Neighbor (ANN)	31
6.1.2	Indexing Strategies: HNSW and IVF	31
6.1.3	GPU vs CPU Trade-offs	32
6.2	Tough Questions	32
6.2.1	How do you scale vector search to billions of documents?	32
6.2.2	How do you optimize TTFT (time to first token)?	33
6.2.3	What are the trade-offs between accuracy and latency in vector search?	33
6.2.4	How do you reduce embedding cost?	34
6.2.5	When should you use caching vs recomputation?	34
7	Data Pipeline & Ingestion	36
7.1	Core Concepts	36
7.1.1	Document Parsing	36
7.1.2	Incremental Updates	36
7.1.3	Data Freshness	37
7.2	Tough Questions	37
7.2.1	How do you handle real-time data updates?	37
7.2.2	How do you re-index without downtime?	38
7.2.3	How do you handle duplicate documents?	38
7.2.4	How do you version embeddings?	39
8	Security & Enterprise RAG	40
8.1	Core Concepts	40
8.1.1	Access Control Models	40
8.1.2	Multi-tenant Isolation	40
8.1.3	PII Filtering	41
8.2	Tough Questions	41
8.2.1	How do you prevent data leakage across users?	41
8.2.2	How do you implement row-level security in RAG?	41
8.2.3	How do you handle sensitive documents?	42
8.2.4	How do you audit RAG outputs?	43
9	Agentic RAG	45
9.1	Core Concepts	45
9.1.1	Planner – Executor – Verifier	45
9.1.2	LangGraph and CrewAI	45
9.1.3	Tool Calling in Agents	46

9.2	Tough Questions	46
9.2.1	When do you use agentic RAG vs simple RAG?	46
9.2.2	How do you avoid infinite loops in agents?	47
9.2.3	How do you evaluate agent performance?	48
9.2.4	How do you orchestrate multiple tools?	48
10	Prompt Engineering for RAG	50
10.1	Core Concepts	50
10.1.1	Context Injection Patterns	50
10.1.2	Structured Outputs	51
10.2	Tough Questions	51
10.2.1	How do you prevent context overflow?	51
10.2.2	How do you structure prompts for grounding?	52
10.2.3	How do you handle conflicting documents?	53
10.2.4	How do you ensure deterministic outputs?	53
11	Observability & Monitoring	55
11.1	Core Concepts	55
11.1.1	What to Log	55
11.1.2	Tracing Tools	55
11.1.3	Drift Detection	56
11.2	Tough Questions	56
11.2.1	What logs are critical in a RAG system?	56
11.2.2	How do you debug bad answers?	56
11.2.3	How do you monitor hallucination in production?	57
11.2.4	How do you implement user feedback loops?	58
12	Deployment & LLMOps	59
12.1	Core Concepts	59
12.1.1	CI/CD for RAG	59
12.1.2	Model Versioning	59
12.1.3	A/B Testing and Canary Deployment	60
12.2	Tough Questions	60
12.2.1	How do you deploy RAG at scale?	60
12.2.2	How do you roll back a bad model?	61
12.2.3	How do you test RAG systems before release?	61
12.2.4	How do you handle multi-region deployment?	62
13	LLM Inference Optimization	64

13.1	KV Cache and Inference Phases	64
13.1.1	Prefill vs Decode Phase	64
13.1.2	KV Cache	64
13.1.3	Paged Attention (vLLM)	65
13.1.4	Continuous Batching	65
13.2	Tough Questions	65
13.2.1	How would you optimize prompt token recomputation across requests?	65
13.2.2	What is KV Cache and why is it critical for LLM serving?	66
13.2.3	How does paged attention work?	66
13.2.4	What techniques optimize GPU memory usage for LLM serving?	67
13.2.5	How do continuous batching and dynamic batching improve throughput?	67
13.2.6	How do you design a scalable LLM inference service on Kubernetes?	68
20	Top System Design Questions & Answers	70
13.2.7	How do you build a robust RAG system end to end?	70
13.2.8	What folder structure would you use for a production RAG system?	71
13.2.9	How do you choose the right RAG framework?	72
13.2.10	How do you design the ingestion and indexing pipeline?	72
13.2.11	How do you design retrieval for high recall and precision?	73
13.2.12	How do you structure the generation layer for grounded answers?	73
13.2.13	How do you design an evaluation framework for RAG?	74
13.2.14	How do you design observability for a RAG platform?	74
13.2.15	How do you improve RAG performance and latency?	75
13.2.16	How do you integrate RAG with cloud infrastructure?	75
13.2.17	What are all the major ways to reduce hallucination in RAG?	75
13.2.18	What caching methodology do you use in production RAG?	76
13.2.19	How do you design memory for conversational RAG?	76
13.2.20	How do you manage context windows effectively?	77
13.2.21	How do you reduce token cost in a large-scale RAG system?	77
13.2.22	How do you secure a multi-tenant RAG system?	78
13.2.23	How do you handle real-time updates and re-indexing?	78
13.2.24	How do you choose the right models across the RAG stack?	78
13.2.25	How do you deploy and roll back a production RAG system safely?	79
13.2.26	What does a strong end-to-end API contract look like for RAG?	79
13.2.27	How do you test a robust RAG system end to end?	80
13.2.28	Give me the concise interview answer for a production-grade RAG system	80

RAG & AI Systems Challenge Trends	81
13.3 Retrieval Quality	81
13.3.1 Why is retrieval quality still hard in production RAG systems? . . .	82
13.3.2 How do you mitigate semantic mismatch, chunking errors, and multi-hop retrieval failures?	82
13.4 Context and Memory	83
13.4.1 Why are context windows and memory still hard problems in RAG?	83
13.4.2 How do you design context assembly and persistent memory safely?	83
13.5 Data Quality and Freshness	84
13.5.1 Why are stale, noisy, and heterogeneous knowledge bases so difficult to manage?	84
13.5.2 How do you handle contradictory documents and mixed structured-unstructured data?	85
13.6 Faithfulness and Hallucination	85
13.6.1 Why do hallucinations persist even when the system retrieves the right context?	85
13.6.2 How do you improve attribution accuracy and confidence calibration?	86
13.7 Evaluation and Observability	86
13.7.1 Why are RAG evaluation and observability still unsolved in practice?	86
13.7.2 How do you manage latency-versus-accuracy tradeoffs and detect silent failures?	87
13.8 Security and Trust	87
13.8.1 What are the main security and trust risks in enterprise RAG? . . .	88
13.9 Architecture and Scaling	88
13.9.1 What architecture and scaling trends are shaping modern RAG systems?	88
13.9.2 How do you control cost at scale without breaking quality?	89
Quick Reference: All Questions Index	90

Preface

This guide is your complete preparation resource for AI engineering interviews, with an emphasis on **Retrieval-Augmented Generation (RAG)**, LLM system design, and production AI infrastructure. It covers the core topic areas that regularly appear in senior-level interviews, from foundational retrieval concepts to enterprise-scale agentic systems and modern inference optimization. This edition also adds a dedicated set of twenty mostly asked system design questions and answers focused on robust RAG architecture, framework choice, observability, performance, cloud integration, hallucination reduction, caching, memory, context management, and low-cost token strategy.

Each chapter presents core concepts followed by the toughest interview questions with complete sample answers and production-ready code examples. The 2026 edition expands the chapter introductions so you can understand not only what to say in an interview, but also why each concept matters in production settings.

The structure of the book is intentionally layered. Early chapters build the retrieval and architecture foundation, middle chapters cover evaluation, reliability, security, and performance, and later chapters address deployment, observability, agentic workflows, and inference optimization. This sequencing helps you move from conceptual clarity to production-level design reasoning.

How to use this book: Read sequentially for full coverage, or jump directly to any question via the Table of Contents. Code samples use a light background for comfortable reading. Each [Q] box contains an exact interview question; each Sample Answer box contains a concise, interview-ready response.

— *Lamhot Siagian*

RAG Fundamentals

Chapter Overview — This chapter establishes the mental model for Retrieval-Augmented Generation by separating retrieval quality, grounding quality, and answer quality. In interviews, strong candidates explain when RAG is better than fine-tuning, when long-context prompting is sufficient, and how chunking, embeddings, and similarity functions affect downstream reliability. Master this chapter first because later design, evaluation, and scaling decisions all depend on these foundations.

1.1 Core Concepts

1.1.1 RAG vs Fine-tuning vs Long-context LLMs

Approach	Best For	Limitations
RAG	Dynamic knowledge, grounding, citations	Retrieval latency, pipeline complexity
Fine-tuning	Style, tone, task format, domain-specific behavior	Expensive, cannot update knowledge easily
Long-context LLM	Short-lived context (200K tokens)	Cost, slow, no fresh knowledge

1.1.2 Embeddings: Dense vs Sparse vs Hybrid

Embedding Types — **Dense embeddings:** Neural network maps text to a continuous vector. Captures semantic meaning; handles synonyms. Examples: `text-embedding-3-large`, BGE, E5.

Sparse embeddings: Term-frequency representation (BM25, TF-IDF). Exact keyword matching. Fast and interpretable.

Hybrid: Combine both via Reciprocal Rank Fusion (RRF). Best precision and recall.

1.1.3 Chunking Strategies

- **Fixed-size:** Split at N characters with overlap. Simple, predictable. Risk: mid-sentence splits.
- **Recursive character:** Split on ["\n\n", "\n", ". "] in priority order. Best default.
- **Semantic:** Embed sentences; split where similarity drops. Coherent chunks; slower.
- **Document-aware:** Parse Markdown/HTML structure; keep sections intact.
- **Parent-child:** Small chunks for retrieval, large parent returned for context.

1.1.4 Vector Databases

DB	Strengths	Index Type	Use Case
FAISS	In-memory, very fast	HNSW, IVF	Research, single node
Pinecone	Managed, serverless	Proprietary	SaaS, low ops
Chroma	Easy local dev	HNSW	Prototypes
Weaviate	Hybrid search native	HNSW + BM25	Production enterprise
Qdrant	Fast, Rust-native	HNSW	High throughput
pgvector	PostgreSQL extension	IVFFlat	Existing PG users

1.1.5 Similarity Search: Cosine vs Dot Product

$$\text{Cosine Similarity} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}, \quad \text{Dot Product} = \mathbf{a} \cdot \mathbf{b}$$

Use cosine when vectors are **not normalized** (removes magnitude bias). Use dot product when vectors are L2-normalized (equivalent, but faster).

1.2 Tough Questions

1.2.1 Why use RAG instead of fine-tuning? When does RAG fail?

[Q] **Interview Question** — Why use RAG instead of fine-tuning? And when does RAG actually fail?

Sample Answer — **RAG over fine-tuning when:**

- Knowledge changes frequently (news, documentation, product catalog)
- You need citations and traceable sources
- Data is too large or sensitive to train on
- Faster iteration required (no retraining)

RAG fails when:

1. **Retrieval fails:** Wrong chunks returned; answer is not in the index
2. **Chunk size mismatch:** Answer spans multiple chunks; none retrieved alone is sufficient
3. **Semantic gap:** Query language differs from document language (jargon, synonyms)
4. **Multi-hop reasoning:** Answer requires combining facts across many documents
5. **Temporal:** Index is stale; retrieved docs are outdated
6. **LLM ignores context:** LLM uses parametric knowledge instead of retrieved context

Fix strategy: Evaluate retrieval and generation *separately*. If retrieval metrics (Recall@K) are low, fix chunking and embedding. If faithfulness is low despite good retrieval, fix the prompt.

1.2.2 How do embedding models impact retrieval quality?

[Q] **Interview Question** — How do embedding models impact retrieval quality? Which would you choose for a domain-specific system?

Sample Answer — Embedding model choice is the *single biggest lever* on retrieval quality. Key factors:

- **Dimensionality:** Higher dims (1536 vs 384) generally more expressive, but slower and costlier
- **Training data:** General-purpose vs domain-trained (legal-BERT, BioMedBERT)
- **Pooling strategy:** CLS token, mean pooling, weighted mean

- **Max context length:** Some models cap at 512 tokens; chunk accordingly

Domain-specific approach:

1. Start with `text-embedding-3-large` or `BGE-large` as baseline
2. Evaluate on your domain test set using MTEB leaderboard metrics
3. If recall is poor, fine-tune embedding model on domain triplets (query, positive, hard negative)
4. Use matryoshka representation learning (MRL) embeddings if you need dimension flexibility

1.2.3 What happens if chunk size is too large or too small?

[Q] **Interview Question** — What are the consequences of incorrect chunk size? How do you tune it?

Sample Answer — Too small (e.g., 50–100 tokens):

- Each chunk lacks context; embeddings are noisy
- Answer may span multiple chunks; none is individually sufficient
- Higher retrieval cost (more chunks to index and search)

Too large (e.g., 2000+ tokens):

- Embedding averages over too much content; precision drops
- Irrelevant content dilutes the relevant signal
- Larger context window needed in generation phase

Tuning approach:

1. Start at 512 tokens with 64-token overlap
2. Measure Context Precision@5 (how much retrieved content is relevant)
3. Grid-search chunk sizes on a labeled eval set
4. Use parent-child chunking: 400-token child for retrieval, 1600-token parent for generation

1.2.4 Explain cosine similarity vs inner product in high-dimensional space

[Q] **Interview Question** — What is the difference between cosine similarity and inner product? When does each fail in high-dim space?

Sample Answer — **Cosine similarity** normalizes by vector magnitude – it measures *angle* only. Two vectors with the same direction but different magnitudes are treated as identical. This removes the influence of document length/quantity.

Dot product (inner product) measures both direction and magnitude. A long document with many relevant terms scores higher even at the same angular distance.

High-dimensional considerations:

- The “curse of dimensionality”: in very high dims, all pairwise distances converge; discrimination weakens
- Cosine similarity concentrates around values of 0.7–0.9 for all pairs; use relative ranking, not absolute thresholds
- **Practical advice:** Normalize all embeddings (L2-norm) at index time; then dot product = cosine similarity at zero extra cost

1.2.5 How do you handle domain-specific vocabulary?

[Q] Interview Question — How would you handle a RAG system where users query with domain-specific jargon that the embedding model has never seen?

Sample Answer —

1. **Query expansion:** Use an LLM to rewrite the query using both jargon and plain-language equivalents before embedding
2. **BM25 hybrid:** Exact keyword matching via BM25 catches jargon even when embeddings fail
3. **Synonym injection:** Maintain a domain glossary; expand acronyms at both index and query time
4. **Fine-tune embeddings:** Create contrastive training pairs from domain query logs (jargon query, relevant passage)
5. **Metadata filtering:** Tag documents with controlled vocabulary; filter before semantic search
6. **Sparse+dense:** SPLADE generates learned sparse representations that better handle rare terms than dense alone

Advanced Retrieval Design

Chapter Overview — This chapter moves from basic retrieval to production-grade search design. The central interview theme is that retrieval quality is rarely solved by a single index; instead, strong systems combine lexical search, dense retrieval, query understanding, and re-ranking to maximize precision under latency constraints. Use this chapter to explain trade-offs clearly rather than describing retrieval as a generic vector lookup.

2.1 Core Concepts

2.1.1 Hybrid Retrieval: BM25 + Vector Search

Hybrid retrieval combines dense semantic search with sparse BM25 keyword search, merged via Reciprocal Rank Fusion (RRF):

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}, \quad k = 60$$

```
1 from langchain.retrievers import EnsembleRetriever
2 from langchain_community.retrievers import BM25Retriever
3
4 # Sparse retriever (BM25)
5 bm25 = BM25Retriever.from_documents(docs)
6 bm25.k = 10
```



```
7
8 # Dense retriever (vector store)
9 dense = vectorstore.as_retriever(search_kwargs={"k": 10})
10
11 # Ensemble via Reciprocal Rank Fusion
12 ensemble = EnsembleRetriever(
13     retrievers=[bm25, dense],
14     weights=[0.35, 0.65] # Tune on your eval set
15 )
16 results = ensemble.invoke("transformer attention mechanism")
```

Listing 2.1: Hybrid Retrieval with RRF

2.1.2 Query Rewriting and Expansion

Three key techniques:

- **Multi-query**: Generate 3–5 reformulations; union of results
- **HyDE**: Generate a hypothetical answer; embed the answer (closer to document distribution)
- **Step-back**: Ask a more abstract form; retrieve foundational context

2.1.3 Re-ranking Pipeline

- **Bi-encoder** (first stage): Fast; encodes query and doc independently
- **Cross-encoder** (re-rank stage): Slow; attends query and doc together; much higher precision
- **LLM-based**: Use an LLM to score relevance; expensive but highest quality

2.1.4 Multi-hop Retrieval

For questions requiring facts from multiple documents, use iterative retrieval:

1. Retrieve for original query
2. Extract entities/claims from retrieved docs
3. Generate follow-up queries for missing facts
4. Re-retrieve and synthesize

2.2 Tough Questions

2.2.1 Design a hybrid retrieval system end-to-end

[Q] **Interview Question** — Design a hybrid retrieval system from scratch. Walk me through every component and decision.

Sample Answer — **End-to-end hybrid retrieval architecture:**

1. Indexing:

- Parse documents (Unstructured.io for PDF/DOCX)
- Chunk: recursive splitter, 512 tokens, 64-token overlap
- Embed with BGE-large-en (dense, 1024 dims)
- Generate sparse BM25 index (Elasticsearch or Weaviate native)
- Store metadata: doc_id, source, section, timestamp

2. Query time:

- Rewrite query (LLM): expand acronyms, add synonyms
- Dense retrieval: top-20 candidates from vector index
- Sparse retrieval: top-20 candidates from BM25
- Merge: RRF with weights tuned on eval set (typically 0.3/0.7 sparse/dense)
- Re-rank top-20 with cross-encoder: return top-5

3. Quality controls:

- Similarity threshold: drop results below 0.65 cosine similarity
- Max context: trim to fit LLM context window
- Deduplication: remove near-duplicate chunks

2.2.2 Why is BM25 + vector better than vector alone?

[Q] **Interview Question** — Why is hybrid BM25 + vector search better than vector-only retrieval?

Sample Answer — **Vector-only failure modes:**

- **Rare terms:** Embedding model averages over vocabulary; rare/new terms get poor representations
- **Exact match queries:** “invoice number INV-2024-0938” needs exact matching, not semantic proximity
- **Acronyms:** “CEO” and “Chief Executive Officer” may not be near each other in embedding space